

Priority Inheritance Protocol for Real-Time Resource Synchronization

Nnaemeka Nnachi-Egwu
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
nnaemeka.nnachi-egwu@stud.hshl.de

Abstract—Shared resources in real-time systems create the priority inversion problem: a high-priority task can be blocked by a lower-priority task for an unbounded duration, invalidating schedulability guarantees. The Priority Inheritance Protocol (PIP), introduced by Sha, Rajkumar, and Lehoczky [1], addresses this by dynamically boosting a resource holder’s priority to match that of any blocked task. This paper presents PIP’s formal model, its bounded-inversion guarantee (Theorem 7.2 [2]), and its limitations (deadlock susceptibility, chained blocking). A verified UPPAAL model and C++ simulator demonstrate the protocol’s behaviour on a canonical three-task example. The schedulability analysis uses both the Liu-Layland utilization test and Response-Time Analysis with blocking terms. PIP is evaluated against four alternative protocols and its applicability to embedded systems contexts is assessed.

Index Terms—priority inversion, priority inheritance, real-time scheduling, resource access protocols, semaphores, UPPAAL, schedulability analysis

I. INTRODUCTION

A real-time computer system is one in which the correctness of behaviour depends not only on the logical results of computation but also on the *physical instant* at which those results are produced [3]. In this model, a result delivered after its deadline is not merely late; it is logically incorrect. Real-time systems must therefore provide timing guarantees under all operating conditions, including worst-case resource contention.

Shared exclusive resources introduce the *priority inversion* problem: a high-priority task can be blocked waiting for a resource held by a lower-priority task, while medium-priority tasks are free to run, extending the blockage without bound [2]. This violates the guarantee on which rate-monotonic and earliest-deadline-first scheduling theory is built; the Earliest Deadline First algorithm is optimal only when there is no resource competition [3].

The Priority Inheritance Protocol (PIP) [1] bounds this inversion by letting a resource holder temporarily inherit the priority of any task it blocks. This paper explains PIP’s design rationale, formal properties, practical limitations, and its place among the five standard resource access protocols in Buttazzo Chapter 7 [2].

II. BACKGROUND: TASK MODEL AND PROBLEM

A. Task Parameters

Following [2] §7.3, each periodic task τ_i is characterised by: nominal priority P_i (assigned by Rate-Monotonic or Deadline-

Monotonic rules, lower number = higher priority), worst-case computation time C_i , period T_i , and relative deadline D_i . Active (dynamic) priority p_i may deviate from P_i under a resource access protocol.

B. Semaphores and Critical Sections

Exclusive resources are protected by binary semaphores. A critical section $z_{i,k}$ of task τ_i on semaphore S_k has duration $\delta_{i,k}$; the collection of all its critical sections is $Z_i = \{z_{i,k}\}$. The $\text{wait}(S_k) / \text{signal}(S_k)$ primitives enforce mutual exclusion. A task blocked on a busy semaphore moves to the WAITING state; the three-state model (READY, RUNNING, WAITING) is taught in the course slides (RTS Introduction, slide 31).

C. Priority Inversion

Without a resource access protocol, consider three tasks $\tau_1 \succ \tau_2 \succ \tau_3$ in priority (Fig. 7.4 in [2]):

- 1) τ_3 acquires semaphore S_R and enters its critical section.
- 2) τ_1 (highest priority) is released and immediately preempts τ_3 . τ_1 calls $\text{wait}(S_R)$ but S_R is held by τ_3 , so τ_1 blocks.
- 3) τ_2 (which does not use S_R) preempts τ_3 .
- 4) τ_1 remains blocked for the entire duration of τ_2 ’s execution, even though τ_2 has lower priority than τ_1 .

If any number of medium-priority tasks arrive during this window, the blocking of τ_1 is *unbounded*. This is the priority inversion problem.

III. PROTOCOL LANDSCAPE

Table I summarises the five resource access protocols in Buttazzo Chapter 7 [2].

NPP (Non-Preemptive Protocol): tasks run non-preemptively in critical sections; single blocking, but over-conservative. **HLP/IPC** (Highest Locker Priority / Immediate Priority Ceiling): tasks inherit the ceiling of the semaphore at entry time; single blocking but requires ceiling declarations. **PIP**: defers blocking to actual access time, no code change required (transparent), bounded inversion, but no deadlock prevention. **PCP** (Priority Ceiling Protocol, also [1]): adds lock-grant rule eliminating deadlock and chained blocking. **SRP** (Stack Resource Policy [4]): extends PCP to multi-unit resources and EDF scheduling, enables stack sharing.

TABLE I
RESOURCE ACCESS PROTOCOL COMPARISON (BUTTAZZO TABLE 7.5, P. 248)

Property	NPP	HLP	PIP	PCP	SRP
Deadlock-free	✓	✓	–	✓	✓
Chained blocking	–	–	Yes	–	–
Transparent	✓	–	✓	–	–
Max blockings	1	1	α_i	1	1
Implementation	Easy	Easy	Hard	Medium	Easy
Dynamic prio.	–	–	–	–	✓

IV. PRIORITY INHERITANCE PROTOCOL

A. Algorithm

PIP [1] is defined via two primitives (Buttazzo §7.6.4, pp. 224–225), reproduced here with slight notation adjustments:

pi_wait(S_k):

- 1) If S_k is free, lock it and assign $S_k.\text{holder} \leftarrow \tau_{exe}$.
- 2) Otherwise, add τ_{exe} to S_k 's wait queue and set τ_{exe} to **BLOCKED**. Let $\tau_j = S_k.\text{holder}$; apply inheritance:

$$p_j(R_k) = \max\{P_j, \max_h\{P_h \mid \tau_h \text{ blocked on } R_k\}\}. \quad (1)$$

If τ_j is itself blocked on another semaphore S_m , apply the rule transitively to $S_m.\text{holder}$.

pi_signal(S_k):

- 1) Unlock S_k .
- 2) Wake the highest-priority task in S_k 's wait queue (if any) and grant it the lock.
- 3) Restore τ_j 's active priority to $\max\{P_j, \max_h\{p_h \mid \tau_h \text{ still blocked by } \tau_j\}\}$.
- 4) If $p_j < p^*$ (highest ready task), perform a context switch.

Inheritance is *transitive*: if τ_3 blocks τ_2 which blocks τ_1 , then τ_3 inherits P_1 via τ_2 (§7.6.1, p. 215, Lemma 7.2 requires nested critical sections for transitivity to arise).

B. Push-Through Blocking

A key consequence of PIP is *push-through blocking* (Buttazzo Lemma 7.1, p. 218): if τ_j (low priority) holds S_R and blocks τ_i (high priority), then τ_j inherits P_i and thereby prevents any intermediate-priority task from preempting it. Lemma 7.1 identifies the exact condition: semaphore S_k causes push-through blocking to τ_i if and only if there exists a task with priority greater than P_i using S_k and a task with priority less than P_i using S_k .

C. Blocking Time Bound

Let l_i be the number of distinct lower-priority tasks that can block τ_i , and s_i the number of distinct semaphores accessed by both τ_i and lower-priority tasks. Then (Theorem 7.2 [1], Buttazzo p. 219):

$$\alpha_i = \min(l_i, s_i). \quad (2)$$

τ_i is blocked by at most α_i critical sections. The worst-case blocking time is computed using the algorithm in Fig. 7.11 [2] ($\mathcal{O}(mn^2)$):

$$B_i^l = \sum_{j>i} \sum_k \delta_{j,k} \cdot \mathcal{K}[\text{block}] \quad (3)$$

$$B_i^s = (\text{sum by semaphore}) \quad (4)$$

$$B_i = \min(B_i^l, B_i^s). \quad (5)$$

This bound is *not tight*: B_i^l may sum critical sections guarded by the same semaphore, which Lemma 7.4 shows cannot both block simultaneously (p. 222).

D. Schedulability

With blocking terms, the Liu-Layland sufficient test (Buttazzo Eq. 7.19) becomes:

$$\forall i: \sum_{h<i} \frac{C_h}{T_h} + \frac{C_i + B_i}{T_i} \leq i(2^{1/i} - 1). \quad (6)$$

When this test is inconclusive, the exact Response-Time Analysis (Eq. 7.22) is applied iteratively:

$$R_i = C_i + B_i + \sum_{h<i} \left\lceil \frac{R_i}{T_h} \right\rceil C_h. \quad (7)$$

E. Limitations

No deadlock prevention. If two tasks acquire two semaphores in reverse order, a lock cycle can form (Fig. 7.13 [2], p. 226): PCP's lock-grant rule is needed to prevent this.

Chained blocking. If τ_i accesses n distinct semaphores each held by a different lower-priority task, it can be blocked n times (Fig. 7.12, p. 225); the number is bounded by α_i from (2), but PCP limits this to exactly one.

Kernel complexity. The transitive chain traversal in **pi_wait** adds latency inside an atomic kernel section; Buttazzo rates PIP's implementation as "hard" — the only protocol with that rating (Table 7.5, p. 248).

V. UPPAAL VERIFICATION

A. Model Structure

A UPPAAL timed-automata model (based on [5]) was constructed with four components: (1) a parametric Task template instantiated for τ_1, τ_2, τ_3 , each with locations **IDLE**, **READY**, **RUNNING**, **CRITICALSECTION**, **BLOCKED**; (2) a **ResourceManager** automaton with locations **FREE**, **HELD**,

and committed GRANTING, encoding the binary semaphore S_R with PIP inheritance logic (Eq. 1); (3) an Observer process for property monitoring.

Shared global variables encode active priorities (`int[1,9] act_prio[3]`), the semaphore holder (`int[-1,2] holder`), blocking duration counters (`int[0,20] block_dur[3]`), and observer flags (`bool blocking_tau1`). Synchronisation channels `lock_req[N]!`, `lock_ack[N]!`, `unlock_rel[N]!` coordinate lock operations between tasks and the resource manager. The key inheritance transition in `ResourceManager` fires on `lock_req[0]?` while `holder==2` (i.e., τ_3 holds the lock when τ_1 requests it), and immediately applies the assignment

```
act_prio[2] := (P1 < act_prio[2]) ? P1
: act_prio[2]
```

implementing Eq. (1) atomically.

B. Properties Verified

The five verification queries target distinct PIP correctness properties.

P1 (Deadlock-free):

```
A[] not deadlock
```

No lock cycle can form with a single resource and consistent acquisition order; this query would *fail* if two resources were acquired in reverse order (Fig. 7.13 [2]).

P2 (No unbounded inversion):

```
A[] (blocking_tau1 imply not
(tau2.Running and not blocked_tau2))
```

When τ_1 is blocked on S_R , τ_2 cannot run freely — verifying push-through blocking (Lemma 7.1). An *invariant*: must hold in every reachable state.

P3 (Blocking within bound):

```
A[] (block_dur[0]<=4 and
block_dur[1]<=4
and block_dur[2]==0)
```

Observed blocking stays within $B_1=4$, $B_2=4$, $B_3=0$, linking the formal model to the schedulability analysis.

P4 (Inheritance reachable):

```
E<> (act_prio[2]==1)
```

There exists a reachable state where τ_3 holds inherited priority P_1 , confirming the inheritance mechanism fires (non-vacuous witness).

P5 (Priority restored):

```
A[] (rm.Free imply act_prio[2]==3)
```

Whenever S_R is free, τ_3 's priority equals P_3 , verifying `restore_prio` correctly undoes every inheritance.

VI. APPLICATION EXAMPLE

A. Task Set

Three periodic tasks are scheduled by Rate-Monotonic (shorter period = higher priority): τ_1 ($C=1, T=6, D=6$, prio 1, uses S_R , CS len 1), τ_2 ($C=2, T=8, D=8$, prio 2, no shared resource), τ_3 ($C=4, T=24, D=24$, prio 3, uses S_R , CS len 4). The semaphore ceiling $C(S_R) = P_1$ (highest priority using S_R).

B. Priority Inversion Trace (Without PIP)

τ_3 is released at $t=0$ and acquires S_R immediately; at $t=2$, τ_1 and τ_2 are released. τ_1 preempts τ_3 , calls `pi_wait( $S_R$ )`, and blocks (S_R held by τ_3 , which retains priority P_3). τ_2 (priority P_2) preempts τ_3 at $t=2$. τ_1 remains blocked for the entire duration of τ_2 's execution (2 units), plus τ_3 's remaining CS (2 units): *total blocking = 4 units*, of which 2 units are due to the lower-priority task τ_2 — priority inversion. Table II shows the schedule.

C. PIP Trace (With PIP)

At $t=2$, when τ_1 blocks on S_R , PIP applies Eq. (1): $p_3(S_R) = \max(P_3, P_1) = P_1 = 1$. τ_3 now has priority 1; τ_2 (priority 2) *cannot* preempt it. τ_3 runs uninterrupted until it signals S_R at $t=4$ (2 remaining CS units). τ_3 's priority is restored to P_3 . τ_1 is unblocked and runs to completion. *Blocking = 2 units* — bounded by τ_3 's remaining CS, with no contribution from τ_2 .

D. Blocking Time Computation

With $\delta_{3,R} = 4$ (CS length, continuous-time convention; Buttazzo's discrete formula subtracts 1, giving $\delta_{3,R}-1 = 3$):

B_1 (**direct blocking**): $\gamma_1 = \{Z_{3,R}\}$ since $P_3 < P_1$ and $C(S_R)=P_1 \geq P_1$ (Lemma 7.5). $l_1 = s_1 = 1$, so $\alpha_1 = 1$. $B_1^l = B_1^s = \delta_{3,R} = 4$, hence $B_1 = 4$.

B_2 (**push-through blocking, Lemma 7.1**): S_R is accessed by τ_1 ($P_1 > P_2$) and τ_3 ($P_3 < P_2$), so τ_3 can inherit $P_1 > P_2$, blocking τ_2 . $l_2 = s_2 = 1$, $\alpha_2 = 1$, $B_2 = 4$.

$B_3 = 0$ (τ_3 is lowest-priority).

$$B_1 = \min(B_1^l, B_1^s) = \min(4, 4) = 4$$

$$B_2 = \min(4, 4) = 4$$

$$B_3 = 0$$

E. Schedulability Analysis

$$U = \frac{1}{6} + \frac{2}{8} + \frac{4}{24} = 0.583.$$

Liu-Layland (Eq. 6) is inconclusive for τ_2 ($0.917 > 0.828$; sufficient test only). Response-Time Analysis (Eq. 7) gives exact results:

$$R_1 = 5 \leq D_1 = 6,$$

$$R_2 = 8 \leq D_2 = 8,$$

$$R_3 = 8 \leq D_3 = 24.$$

All tasks are schedulable under RM with PIP. This illustrates Buttazzo's observation that the Liu-Layland test becomes only sufficient in the presence of blocking terms (p. 246).

VII. C++ SIMULATOR

A discrete-time simulator was implemented in C++17 to validate the hand trace and confirm that blocking bounds hold over a full hyperperiod ($\text{LCM}(6, 8, 24)=24$ time units), matching the same task parameters as Section VI.

TABLE II
SCHEDULE TRACE (ONE COLUMN PER TIME UNIT). W = WAITING FOR S_R ; CS = IN CRITICAL SECTION; † = INHERITS P_1 .

Task	0	1	2	3	4	5	6	7	...
<i>Without PIP:</i>									
τ_1	–	–	W	W	W	W	run	–	
τ_2	–	–	run	run	–	–	–	–	
τ_3	CS	CS	–	–	CS	CS	–	–	
<i>With PIP:</i>									
τ_1	–	–	W	W	run	–	–	–	
τ_2	–	–	rdy	rdy	run	run	–	–	
τ_3	CS	CS	CS [†]	CS [†]	–	–	–	–	

A. Design

The core data structures mirror Buttazzo’s notation directly. Each Task struct holds `nom_prio` (nominal, fixed), `act_prio` (active, modified by PIP), `remaining` (computation time left), CS state variables (`has_lock`, `cs_rem`), and blocking metadata (`blk_start`, `max_blk`). The single Semaphore struct tracks `holder` and a wait queue of task indices.

The `inherit_up(holder, req_prio)` function implements Eq. (1) recursively: if the requested priority is higher than the holder’s current active priority, boost the holder and check whether the holder is itself in a wait queue, triggering another `inherit_up` call on that queue’s holder (transitive inheritance, Lemma 7.2). `restore_prio` scans all semaphore wait queues on release and sets the task’s active priority to the maximum of its nominal priority and the highest priority among remaining waiters.

B. Simulation Results

Without PIP: At $t=4$, τ_1 ’s second instance preempts τ_3 and blocks on S_R . Since τ_3 retains its nominal priority P_3 , τ_2 preempts τ_3 when it becomes ready. τ_1 remains blocked for 5 units (inversion window: τ_2 executes, then τ_3 finishes CS). This exceeds the bound $B_1=4$ — consistent with the theory: without a protocol there is no bound. τ_1 misses its deadline ($R_1=7 > D_1=6$).

With PIP: At $t=4$, τ_1 blocks and `inherit_up` immediately raises τ_3 ’s active priority to $P_1=1$. τ_2 (priority $P_2=2$) cannot preempt τ_3 -at- P_1 . τ_3 completes its CS at $t=7$; `restore_prio` sets τ_3 back to $P_3=3$. τ_1 ’s observed blocking: $7-4=3 \leq B_1=4$ ✓. All deadlines met across the full hyperperiod. Simulator output: `taul: observed=3, B1_bound=4 [OK]`.

VIII. CRITICAL EVALUATION

A. Strengths

Bounded inversion. Theorem 7.2 provides a formal upper bound $\alpha_i = \min(l_i, s_i)$ on the number of critical sections blocking τ_i ; unbounded priority inversion is impossible under PIP.

Transparency. PIP requires no change to task code (§7.10, p. 247); `pi_wait/pi_signal` are drop-in replacements for

standard semaphore primitives. This makes PIP the natural choice for POSIX systems (PTHREAD_PRIO_INHERIT [1]) and legacy RTOS migration. The 1997 Mars Pathfinder anomaly was resolved by enabling PTHREAD_PRIO_INHERIT on a VxWorks mutex — a one-line configuration change, no code restructuring.

Schedulability integration. The B_i values feed directly into standard schedulability tests (Eqs. 6–7).

B. Limitations

No deadlock prevention. Two tasks acquiring two semaphores in reverse order create a lock cycle that PIP cannot detect or break (Fig. 7.13 [2]). The fix is either semaphore total-ordering (by ID) or replacement with PCP.

Chained blocking. Multiple sequential semaphore accesses can produce $\alpha_i > 1$ blockings (Fig. 7.12). PCP eliminates this by guaranteeing a single blocking per task.

WCET input sensitivity. Equations 3–5 require exact $\delta_{j,k}$ values (CS WCETs). Abella et al. [6] show that none of the current WCET analysis methods is fully trustworthy on modern hardware; under-estimated WCETs may make the system appear schedulable when it is not.

Uniprocessor scope. Buttazzo Chapter 7 explicitly addresses uniprocessor systems (§7.3). On multiprocessors, priority boosting requires inter-processor interrupts (IPIs) with non-deterministic latency. Gracioli et al. [7] measure such overhead in a real RTOS and show it has no equivalent in Buttazzo’s blocking formula; PIP as defined by [1] does not directly transfer to multiprocessor contexts.

C. Transfer Analysis

PIP is suitable for: single-core RTOS with few shared resources and total semaphore ordering enforced; POSIX-based systems requiring transparent legacy migration; automotive ECUs on classic AUTOSAR single-core platforms. PIP is unsuitable for: multiprocessor/multicore systems; EDF-scheduled systems (use SRP [4]); safety-critical contexts requiring certified deadlock freedom (use PCP).

IX. CONCLUSION

The Priority Inheritance Protocol solves the priority inversion problem by dynamically elevating resource holders’ priorities, bounding τ_i ’s blocking to at most $\alpha_i = \min(l_i, s_i)$ critical

sections (Theorem 7.2). Its transparency allows deployment without code changes, making it attractive for legacy systems and POSIX environments. However, PIP does not prevent deadlocks or chained blocking, both eliminated by the Priority Ceiling Protocol at the cost of transparency. The exact Response-Time Analysis (Eq. 7) is necessary for schedulability decisions when the Liu-Layland test is inconclusive. On multiprocessors, PIP's assumptions break due to IPI latency; this remains an active research direction.

ACKNOWLEDGEMENTS

I thank Prof. Dr. Stefan Henkler for introducing the topic of real-time resource synchronisation and for the structured seminar milestones that guided this work. AI usage during this work is documented in the accompanying AI usage protocol files as required by the seminar protocol.

REFERENCES

- [1] L. Sha, R. Rajkumar, and J. P. Lehoczky, "Priority inheritance protocols: An approach to real-time synchronization," *IEEE Transactions on Computers*, vol. 39, no. 9, pp. 1175–1185, 1990.
- [2] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011.
- [3] H. Kopetz, *Real-Time Systems: Design Principles for Distributed Embedded Applications*, 2nd ed. Springer, 2011.
- [4] T. P. Baker, "Stack-based scheduling of realtime processes," *Real-Time Systems*, vol. 3, no. 1, pp. 67–99, 1991.
- [5] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on Uppaal," in *Formal Methods for the Design of Real-Time Systems (SFM-RT 2004)*, ser. Lecture Notes in Computer Science, vol. 3185. Springer, 2004, pp. 200–236.
- [6] J. Abella, C. Hernandez, E. Quiñones, F. J. Cazorla, P. R. Conmy, M. Azkarate-askasua, J. Perez, E. Mezzetti, and T. Vardanega, "WCET analysis methods: Pitfalls and challenges on their trustworthiness," in *Proceedings of the 10th IEEE International Symposium on Industrial Embedded Systems (SIES 2015)*, 2015.
- [7] G. Gracioli, A. A. Fröhlich, R. Pellizzoni, and S. Fischmeister, "Implementation and evaluation of global and partitioned scheduling in a real-time OS," *Real-Time Systems*, vol. 49, no. 5, pp. 669–714, 2013.